Experiment 1

1(A) Linear search

AIM:

To write a program to search for a given element in an array using linear search

SOURCE CODE:

```
#include <iostream>
using namespace std;

int main () {
   int n, k;
   cout << "Enter no. of elements: ";
   cin >> n;
   int a[n];
   cout << "Enter elements: ";

   for (int i = 0; i < n; i++)
      cin >> a[i];

   cout << "Enter element to search: ";
   cin >> k;

   for (int i = 0; i < n; i++) {
      if (a[i] == k) {
         cout << "Element found at position " << i + 1;
         return 0;
      }
   }

   cout << "Element not found";
   return 0;
}
```

OUTPUT:

```
Enter no. of elements: 8
Enter elements: 12 45 78 3 91 34 66 50
Enter element to search: 34
Element found at position 6
PS C:\Users\vihaangoyal\vs code\C_Plus-ULTRA>
```

Experiment 1

1(B) Finding second largest number

AIM:

To write a program to find second largest element in an array

SOURCE CODE:

```
#include <iostream>
using namespace std;

int main() {
   int n;
   cout << "Enter no. of elements: ";
   cin >> n;
   int a[n];
   cout << "Enter elements: ";
   for (int i = 0; i < n; i++)
      cin >> a[i];

   int largest = a[0], second = a[0];
   for (int i = 1; i < n; i++) {
      if (a[i] > largest) {
         second = largest;
         largest = a[i];
      }
      else if (a[i] > second && a[i] != largest)
         second = a[i];
   }

   cout << "Second largest element is " << second;
   return 0;
}
```

OUTPUT:

```
Enter no. of elements: 8
Enter elements: 12 45 78 3 91 34 66 50
Second largest element is 78
PS C:\Users\vihaangoyal\vs code\C_Plus-ULTRA>
```

Experiment 1

1(C) Count number of even and odd elements

AIM:

To write a program to count the number of even and odd elements present in an array.

SOURCE CODE:

```
#include <iostream>
using namespace std;

int main() {
   int n, even = 0, odd = 0;
   cout << "Enter no. of elements: ";
   cin >> n;
   int a[n];

   cout << "Enter elements: ";
   for (int i = 0; i < n; i++)
      cin >> a[i];

   for (int i = 0; i < n; i++) {
      if (a[i] % 2 == 0)
         even++;
      else
         odd++;
   }

   cout << "Number of even elements: " << even << endl;
   cout << "Number of odd elements: " << odd;

   return 0;
}
```

OUTPUT:

```
Enter no. of elements: 8
Enter elements: 12 45 78 3 91 34 66 50
Number of even elements: 5
Number of odd elements: 3
PS C:\Users\vihaangoyal\vs code\C_Plus-ULTRA>
```

Experiment 1

1(D) Remove Duplicate Elements from an Array

AIM:

To write a program to remove duplicate elements from an array and display the resulting array.

SOURCE CODE:

```
#include <iostream>
using namespace std;

int main() {
   int n;
   cout << "Enter no. of elements: ";
   cin >> n;
   int a[n];

   cout << "Enter elements: ";
   for (int i = 0; i < n; i++)
      cin >> a[i];

   for (int i = 0; i < n; i++) {
      for (int j = i + 1; j < n; j++) {
         if (a[i] == a[j]) {
            for (int k = j; k < n - 1; k++)
               a[k] = a[k + 1];
            n--;
            j--;
         }
      }
   }

   cout << "Array after removing duplicates: ";
   for (int i = 0; i < n; i++)
      cout << a[i] << " ";

   return 0;
}
```

OUTPUT:

```
Enter no. of elements: 10
Enter elements: 4 8 15 8 23 4 42 15 16 23
Array after removing duplicates: 4 8 15 23 42 16
PS C:\Users\vihaangoyal\vs code\C_Plus-ULTRA>
```

Experiment 1

1(E) Insert an Element at a Given Position

AIM:

To write a program to insert an element at a specified position in an array.

SOURCE CODE:

```
#include <iostream>
using namespace std;

int main() {
    int n, k, pos;
    cout << "Enter no. of elements: ";
    cin >> n;
    int a[n + 1];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    cout << "Enter element to insert: ";
    cin >> k;
    cout << "Enter position: ";
    cin >> pos;

    for (int i = n; i >= pos; i--)
        a[i] = a[i - 1];

    a[pos - 1] = k;
    n++;

    cout << "Array after insertion: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

OUTPUT:

```
Enter no. of elements: 6
Enter elements: 10 20 30 40 50 60
Enter element to insert: 99
Enter position: 3
Array after insertion: 10 20 99 30 40 50 60
PS C:\Users\vihaangoyal\vs code\C_Plus-ULTRA>
```

Experiment 1

1(F) Delete an Element from a Given Position

AIM:

To write a program to delete an element from a specified position in an array.

SOURCE CODE:

```
#include <iostream>
using namespace std;

int main() {
    int n, pos;
    cout << "Enter no. of elements: ";
    cin >> n;
    int a[n];

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    cout << "Enter position to delete: ";
    cin >> pos;

    for (int i = pos - 1; i < n - 1; i++)
        a[i] = a[i + 1];

    n--;

    cout << "Array after deletion: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

OUTPUT:

```
Enter no. of elements: 6
Enter elements: 10 20 30 40 50 60
Enter position to delete: 4
Array after deletion: 10 20 30 50 60
PS C:\Users\vihaangoyal\vs code\C_Plus-ULTRA>
```

Experiment 1

1(G) Check Whether an Array is Palindrome

AIM:

Write a program to search for a given element in an array using linear search

SOURCE CODE:

```
#include <iostream>
using namespace std;

int main() {
   int n, pol = 1;
   cout << "Enter no. of elements: ";
   cin >> n;
   int a[n];
   cout << "Enter elements: ";
   for (int i = 0; i < n; i++)
      cin >> a[i];

   for (int i = 0; i < n / 2; i++) {
      if (a[i] != a[n - i - 1]) {
         pol = 0;
         break;
      }
   }

   if (pol)
      cout << "Array is a palindrome";
   else
      cout << "Array is not a palindrome";

   return 0;
}
```

OUTPUT:

```
Enter no. of elements: 7
Enter elements: 5 12 34 99 34 12 5
Array is a palindrome
PS C:\Users\vihaangoyal\vs code\C_Plus-ULTRA>
```

```
Enter no. of elements: 7
Enter elements: 5 12 34 99 34 12 8
Array is not a palindrome
PS C:\Users\vihaangoyal\vs code\C_Plus-ULTRA>
```

Experiment 2

AIM:

Write a menu-driven C program implementing:
Insert at beginning
Insert at end
Insert at a given position
Delete from beginning
Delete from end
Delete a specified element
Display the list

SOURCE CODE:

```
#include <iostream>
#include <cstdlib>
using namespace std;

struct node {
   int data;
   struct node *next;
};

struct node* insert_beg(struct node* head, int info) {
   struct node *temp;
   temp = (struct node*)malloc(sizeof(struct node));
   temp->data = info;
   temp->next = head;
   head = temp;
   return head;
}

struct node* insert_end(struct node* head, int info) {
   struct node *temp, *ptr;
   temp = (struct node*)malloc(sizeof(struct node));
   temp->data = info;
   temp->next = NULL;

   if (head == NULL) {
      head = temp;
      return head;
   }

   ptr = head;
   while (ptr->next != NULL) {
      ptr = ptr->next;
   }

   ptr->next = temp;
   return head;
}

struct node* insert_pos(struct node* head, int info, int pos) {
```

```
    if (pos == 1) {
        return insert_beg(head, info);
    }

    struct node *temp, *ptr;
    temp = (struct node*)malloc(sizeof(struct node));
    temp->data = info;
    ptr = head;

    for (int i = 1; i < pos - 1 && ptr != NULL; i++) {
        ptr = ptr->next;
    }

    if (ptr == NULL) {
        cout << "Invalid position";
        return head;
    }

    temp->next = ptr->next;
    ptr->next = temp;
    return head;
}

struct node* delete_beg(struct node* head) {
    if (head == NULL) {
        cout << "List is empty";
        return head;
    }

    head = head->next;
    return head;
}

struct node* delete_end(struct node* head) {
    if (head == NULL) {
        cout << "List is empty";
        return head;
    }

    if (head->next == NULL) {
        return NULL;
    }

    struct node *ptr;
    ptr = head;

    while (ptr->next->next != NULL) {
        ptr = ptr->next;
    }

    ptr->next = NULL;
    return head;
}
```

```
struct node* delete_element(struct node* head, int info) {
   if (head == NULL) {
      cout << "List is empty";
      return head;
   }

   if (head->data == info) {
      return delete_beg(head);
   }

   struct node *ptr;
   ptr = head;

   while (ptr->next != NULL && ptr->next->data != info) {
      ptr = ptr->next;
   }

   if (ptr->next == NULL) {
      cout << "Element not found";
      return head;
   }

   ptr->next = ptr->next->next;
   return head;
}

void display(struct node* head) {
   if (head == NULL) {
      cout << "List is empty";
      return;
   }

   struct node *temp;
   temp = head;

   while (temp != NULL) {
      cout << temp->data << " ";
      temp = temp->next;
   }
}

int main() {
   struct node *head = NULL;
   int choice, info, pos;

   do {
      cout << "\n\n1. Insert at beginning\n2. Insert at end\n3. Insert at given position\n4.
Delete from beginning\n5. Delete from end\n6. Delete a specified element\n7. Display the
list\n8. Exit\nEnter choice: ";
      cin >> choice;

      switch (choice) {
```

```
            case 1:
                cout << "Enter element: ";
                cin >> info;
                head = insert_beg(head, info);
                break;

            case 2:
                cout << "Enter element: ";
                cin >> info;
                head = insert_end(head, info);
                break;

            case 3:
                cout << "Enter element: ";
                cin >> info;
                cout << "Enter position: ";
                cin >> pos;
                head = insert_pos(head, info, pos);
                break;

            case 4:
                head = delete_beg(head);
                break;

            case 5:
                head = delete_end(head);
                break;

            case 6:
                cout << "Enter element to delete: ";
                cin >> info;
                head = delete_element(head, info);
                break;

            case 7:
                display(head);
                break;

            case 8:
                cout << "Exiting...";
                break;

            default:
                cout << "Invalid choice";
        }

    } while (choice != 8);

    return 0;
}
```

OUTPUT:

```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 1
Enter element: 1
```

```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 1
Enter element: 2
```

```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 2
Enter element: 3
```

```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 3
Enter element: 4
Enter position: 2
```

```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 7
2 4 1 3
```

```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 4
```

```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 5
```

```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 1
Enter element: 5
```

```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 6
Enter element to delete: 4
```


```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 5
```

```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 1
Enter element: 5
```

```
1. Insert at beginning
2. Insert at end
3. Insert at given position
4. Delete from beginning
5. Delete from end
6. Delete a specified element
7. Display the list
8. Exit
Enter choice: 6
Enter element to delete: 4
```

Experiment 3

AIM:

Create a linked list with nodes having information about a student and perform:
a. Insert a new node at the specified position.
b. Delete a node with the roll number of a student specified.
c. Reversal of that linked list.

SOURCE CODE:

```
#include <iostream>
#include <cstdlib>
using namespace std;

struct node {
   int roll;
   char name[50];
   struct node *next;
};

struct node* insert_pos(struct node* head, int roll, char name[], int pos) {
   struct node *temp, *ptr;
   temp = (struct node*)malloc(sizeof(struct node));
   temp->roll = roll;
   int i = 0;
   while (name[i] != '\0') {
      temp->name[i] = name[i];
      i++;
   }

   temp->name[i] = '\0';
   if (pos == 1) {
      temp->next = head;
      head = temp;
      return head;
   }

   ptr = head;
   for (int i = 1; i < pos - 1 && ptr != NULL; i++) {
      ptr = ptr->next;
   }

   if (ptr == NULL) {
      cout << "Invalid position";
      return head;
   }

   temp->next = ptr->next;
   ptr->next = temp;
   return head;
}
```

```
struct node* delete_roll(struct node* head, int roll) {
   if (head == NULL) {
      cout << "List is empty";
      return head;
   }

   if (head->roll == roll) {
      head = head->next;
      return head;
   }

   struct node *ptr;
   ptr = head;
   while (ptr->next != NULL && ptr->next->roll != roll) {
      ptr = ptr->next;
   }

   if (ptr->next == NULL) {
      cout << "Roll number not found";
      return head;
   }
   ptr->next = ptr->next->next;
   return head;
}

struct node* reverse(struct node* head) {
   struct node *prev, *ptr, *next;
   prev = NULL;
   ptr = head;
   while (ptr != NULL) {
      next = ptr->next;
      ptr->next = prev;
      prev = ptr;
      ptr = next;
   }

   head = prev;
   return head;
}

void display(struct node* head) {
   if (head == NULL) {
      cout << "List is empty";
      return;
   }

   struct node *temp;
   temp = head;
   while (temp != NULL) {
      cout << "\nRoll No.: " << temp->roll;
      cout << "\nName: " << temp->name;
      temp = temp->next;
   }
```

```
}

int main() {
   struct node *head = NULL;
   int choice, roll, pos;
   char name[50];
   do {
      cout << "\n\n1. Insert at specified position\n2. Delete using roll number\n3. Reverse the
list\n4. Display the list\n5. Exit\nEnter choice: ";
      cin >> choice;
      switch (choice) {
         case 1:
            cout << "Enter roll number: ";
            cin >> roll;
            cout << "Enter name: ";
            cin >> name;
            cout << "Enter position: ";
            cin >> pos;
            head = insert_pos(head, roll, name, pos);
            break;

         case 2:
            cout << "Enter roll number to delete: ";
            cin >> roll;
            head = delete_roll(head, roll);
            break;

         case 3:
            head = reverse(head);
            cout << "List reversed";
            break;

         case 4:
            display(head);
            break;

         case 5:
            cout << "Exiting...";
            break;

         default:
            cout << "Invalid choice";
      }

   } while (choice != 5);

   return 0;
}
```

OUTPUT:

```
1. Insert at specified position
2. Delete using roll number
3. Reverse the list
4. Display the list
5. Exit
Enter choice: 1
Enter roll number: 3
Enter name: shrut
Enter position: 1


1. Insert at specified position
2. Delete using roll number
3. Reverse the list
4. Display the list
5. Exit
Enter choice: 1
Enter roll number: 1
Enter name: bruce
Enter position: 1


1. Insert at specified position
2. Delete using roll number
3. Reverse the list
4. Display the list
5. Exit
Enter choice: 1
Enter roll number: 2
Enter name: tony
Enter position: 2
```

```
1. Insert at specified position
2. Delete using roll number
3. Reverse the list
4. Display the list
5. Exit
Enter choice: 4

Roll No.: 1
Name: bruce
Roll No.: 2
Name: tony
Roll No.: 3
Name: shrut

1. Insert at specified position
2. Delete using roll number
3. Reverse the list
4. Display the list
5. Exit
Enter choice: 3
List reversed

1. Insert at specified position
2. Delete using roll number
3. Reverse the list
4. Display the list
5. Exit
Enter choice: 4

Roll No.: 3
Name: shrut
Roll No.: 2
Name: tony
Roll No.: 1
Name: bruce
```

```
1. Insert at specified position
2. Delete using roll number
3. Reverse the list
4. Display the list
5. Exit
Enter choice: 5
Exiting...
```

Experiment 4

AIM:

Create a doubly linked list with nodes having information about an employee and perform Insertion at front of the doubly linked list and perform deletion at end of that doubly linked list.

SOURCE CODE:

```
#include <iostream>
#include <cstdlib>
using namespace std;

struct node {
   int id;
   char name[50];
   struct node *prev;
   struct node *next;
};

struct node* insert_front(struct node* head, int id, char name[]) {
   struct node *temp;
   temp = (struct node*)malloc(sizeof(struct node));
   temp->id = id;
   int i = 0;
   while (name[i] != '\0') {
      temp->name[i] = name[i];
      i++;
   }

   temp->name[i] = '\0';
   temp->prev = NULL;
   temp->next = head;
   if (head != NULL)
      head->prev = temp;

   head = temp;
   return head;
}

struct node* delete_end(struct node* head) {
   if (head == NULL) {
      cout << "List is empty";
      return head;
   }

   if (head->next == NULL) {
      return NULL;
   }

   struct node *ptr;
   ptr = head;
```

```
    while (ptr->next != NULL)
        ptr = ptr->next;

    ptr->prev->next = NULL;
    return head;
}

void display(struct node* head) {
    if (head == NULL) {
        cout << "List is empty";
        return;
    }

    struct node *temp;
    temp = head;
    while (temp != NULL) {
        cout << "\nEmployee ID: " << temp->id;
        cout << "\nName: " << temp->name;
        temp = temp->next;
    }
}

int main() {
    struct node *head = NULL;
    int choice, id;
    char name[50];

    do {
        cout << "\n\n1. Insert at front\n2. Delete from end\n3. Display the list\n4. Exit\nEnter
choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                cout << "Enter employee ID: ";
                cin >> id;
                cout << "Enter employee name: ";
                cin >> name;
                head = insert_front(head, id, name);
                break;

            case 2:
                head = delete_end(head);
                break;

            case 3:
                display(head);
                break;

            case 4:
                cout << "Exiting...";
                break;

            default:
```

```
            cout << "Invalid choice";
        }

    } while (choice != 4);
    return 0;
}
```

OUTPUT:

```
1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 1
Enter employee ID: 23
Enter employee name: aditya


1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 1
Enter employee ID: 22
Enter employee name: aryan


1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 1
Enter employee ID: 21
Enter employee name: arnav


1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 3
```

```
Employee ID: 21
Name: arnav
Employee ID: 22
Name: aryan
Employee ID: 23
Name: aditya

1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 2


1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 3

Employee ID: 21
Name: arnav
Employee ID: 22
Name: aryan

1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 4
Exiting...
```

Experiment 5

AIM:

To create a circular linked list containing information about a college and perform insertion at the front and deletion at the end.

SOURCE CODE:

```
#include <iostream>
#include <cstdlib>
using namespace std;

struct node {
   char name[50];
   char city[50];
   struct node *next;
};

struct node* insert_front(struct node* head, char name[], char city[]) {
   struct node *temp, *ptr;
   temp = (struct node*)malloc(sizeof(struct node));
   int i = 0;
   while (name[i] != '\0') {
      temp->name[i] = name[i];
      i++;
   }

   temp->name[i] = '\0';
   i = 0;
   while (city[i] != '\0') {
      temp->city[i] = city[i];
      i++;
   }

   temp->city[i] = '\0';
   if (head == NULL) {
      temp->next = temp;
      head = temp;
      return head;
   }

   ptr = head;
   while (ptr->next != head)
      ptr = ptr->next;
   temp->next = head;
   ptr->next = temp;
   head = temp;
   return head;
}

struct node* delete_end(struct node* head) {
   if (head == NULL) {
```

```
        cout << "List is empty";
        return head;
    }

    if (head->next == head) {
        return NULL;
    }

    struct node *ptr;
    ptr = head;
    while (ptr->next->next != head)
        ptr = ptr->next;

    ptr->next = head;
    return head;
}

void display(struct node* head) {
    if (head == NULL) {
        cout << "List is empty";
        return;
    }

    struct node *temp;
    temp = head;
    do {
        cout << "\nCollege Name: " << temp->name;
        cout << "\nCity: " << temp->city;
        temp = temp->next;
    } while (temp != head);
}

int main() {
    struct node *head = NULL;
    int choice;
    char name[50], city[50];
    do {
        cout << "\n\n1. Insert at front\n2. Delete from end\n3. Display the list\n4. Exit\nEnter
choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                cout << "Enter college name: ";
                cin >> name;
                cout << "Enter city: ";
                cin >> city;
                head = insert_front(head, name, city);
                break;

            case 2:
                head = delete_end(head);
                break;
```

```
        case 3:
            display(head);
            break;

        case 4:
            cout << "Exiting...";
            break;

        default:
            cout << "Invalid choice";
    }

  } while (choice != 4);
  return 0;
}
```

OUTPUT:

```
1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 1
Enter college name: iitb
Enter city: mumbai


1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 1
Enter college name: iitd
Enter city: delhi


1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 1
Enter college name: mait
Enter city: delhi


1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 3
```

```
College Name: mait
City: delhi
College Name: iitd
City: delhi
College Name: iitb
City: mumbai

1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 2


1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 3

College Name: mait
City: delhi
College Name: iitd
City: delhi

1. Insert at front
2. Delete from end
3. Display the list
4. Exit
Enter choice: 4
Exiting...
```

Experiment 6

AIM:

To create a stack using a linked list and perform Push, Pop, Peek and Traverse operations.

SOURCE CODE:

```
#include <iostream>
#include <cstdlib>
using namespace std;

struct node {
   int data;
   struct node *next;
};

struct node* push(struct node* top, int data) {
   struct node *temp;
   temp = (struct node*)malloc(sizeof(struct node));
   temp->data = data;
   temp->next = top;
   top = temp;
   return top;
}

struct node* pop(struct node* top) {
   if (top == NULL) {
      cout << "Stack is empty";
      return top;
   }

   top = top->next;
   return top;
}

void peek(struct node* top) {
   if (top == NULL) {
      cout << "Stack is empty";
      return;
   }

   cout << "Top element: " << top->data;
}

void traverse(struct node* top) {
   if (top == NULL) {
      cout << "Stack is empty";
      return;
   }
```

```
    struct node *temp;
    temp = top;
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
}

int main() {
    struct node *top = NULL;
    int choice, data;
    do {
        cout << "\n\n1. Push\n2. Pop\n3. Peek\n4. Traverse\n5. Exit\nEnter choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                cout << "Enter data: ";
                cin >> data;
                top = push(top, data);
                break;

            case 2:
                top = pop(top);
                break;

            case 3:
                peek(top);
                break;

            case 4:
                traverse(top);
                break;

            case 5:
                cout << "Exiting...";
                break;

            default:
                cout << "Invalid choice";
        }
    } while (choice != 5);
    return 0;
}
```

OUTPUT:

```
1. Push
2. Pop
3. Peek
4. Traverse
5. Exit
Enter choice: 1
Enter data: 5


1. Push
2. Pop
3. Peek
4. Traverse
5. Exit
Enter choice: 1
Enter data: 4


1. Push
2. Pop
3. Peek
4. Traverse
5. Exit
Enter choice: 1
Enter data: 3


1. Push
2. Pop
3. Peek
4. Traverse
5. Exit
Enter choice: 3
Top element: 3
```

```
1. Push
2. Pop
3. Peek
4. Traverse
5. Exit
Enter choice: 4
3 4 5

1. Push
2. Pop
3. Peek
4. Traverse
5. Exit
Enter choice: 2


1. Push
2. Pop
3. Peek
4. Traverse
5. Exit
Enter choice: 4
4 5

1. Push
2. Pop
3. Peek
4. Traverse
5. Exit
Enter choice: 5
Exiting...
```

Experiment 7

AIM:

To implement two stacks using a single array.

SOURCE CODE:

```
#include <iostream>
using namespace std;
#define MAX 10

int stack[MAX];
int top1 = -1;
int top2 = MAX;

void push1(int data) {
   if (top1 + 1 == top2) {
      cout << "Stack Overflow";
      return;
   }

   top1++;
   stack[top1] = data;
}

void push2(int data) {
   if (top1 + 1 == top2) {
      cout << "Stack Overflow";
      return;
   }

   top2--;
   stack[top2] = data;
}

void pop1() {
   if (top1 == -1) {
      cout << "Stack 1 is empty";
      return;
   }

   cout << "Deleted: " << stack[top1];
   top1--;
}

void pop2() {
   if (top2 == MAX) {
      cout << "Stack 2 is empty";
      return;
   }

   cout << "Deleted: " << stack[top2];
```

```
    top2++;
}

void display() {
    cout << "\nStack 1: ";
    if (top1 == -1)
        cout << "Empty";
    else {
        for (int i = top1; i >= 0; i--)
            cout << stack[i] << " ";
    }

    cout << "\nStack 2: ";
    if (top2 == MAX)
        cout << "Empty";
    else {
        for (int i = top2; i < MAX; i++)
            cout << stack[i] << " ";
    }
}

int main() {
    int choice, data;
    do {
        cout << "\n\n1. Push in Stack 1\n2. Push in Stack 2\n3. Pop from Stack 1\n4. Pop from
Stack 2\n5. Display\n6. Exit\nEnter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter data: ";
                cin >> data;
                push1(data);
                break;

            case 2:
                cout << "Enter data: ";
                cin >> data;
                push2(data);
                break;

            case 3:
                pop1();
                break;

            case 4:
                pop2();
                break;

            case 5:
                display();
                break;
```

```
        case 6:
            cout << "Exiting...";
            break;

        default:
            cout << "Invalid choice";
    }
  } while (choice != 6);
  return 0;
}
```

OUTPUT:

```
1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 1
Enter data: 3
```

```
1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 2
Enter data: 6
```

```
1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 1
Enter data: 2
```

```
1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 2
Enter data: 7
```

```
1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 1
Enter data: 1
```

```
1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 2
Enter data: 8
```

```
1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 5

Stack 1: 1 2 3
Stack 2: 8 7 6

1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 3
Deleted: 1

1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 4
Deleted: 8

1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 4
```

```
1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 5

Stack 1: 2 3
Stack 2: 6

1. Push in Stack 1
2. Push in Stack 2
3. Pop from Stack 1
4. Pop from Stack 2
5. Display
6. Exit
Enter choice: 6
Exiting...
```

Experiment 8

AIM:

To create a linear queue using a linked list and perform Insert, Delete, and Display operations.

SOURCE CODE:

```
#include <iostream>
#include <cstdlib>
using namespace std;

struct node {
   int data;
   struct node *next;
};

struct node *front = NULL;
struct node *rear = NULL;

void insert(int data) {
   struct node *temp;
   temp = (struct node*)malloc(sizeof(struct node));
   temp->data = data;
   temp->next = NULL;
   if (rear == NULL) {
      front = rear = temp;
      return;
   }

   rear->next = temp;
   rear = temp;
}

void delete_element() {
   if (front == NULL) {
      cout << "Queue is empty";
      return;
   }

   front = front->next;
   if (front == NULL)
      rear = NULL;
}

void display() {
   if (front == NULL) {
      cout << "Queue is empty";
      return;
   }

   struct node *temp;
   temp = front;
```

```
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
}

int main() {
    int choice, data;
    do {
        cout << "\n\n1. Insert\n2. Delete\n3. Display\n4. Exit\nEnter choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                cout << "Enter data: ";
                cin >> data;
                insert(data);
                break;

            case 2:
                delete_element();
                break;

            case 3:
                display();
                break;

            case 4:
                cout << "Exiting...";
                break;

            default:
                cout << "Invalid choice";
        }
    } while (choice != 4);
    return 0;
}
```

OUTPUT:

```
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter data: 6


1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter data: 7


1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter data: 8


1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
6 7 8
```

```
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 2


1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 2


1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
8

1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 4
Exiting...
```